

ASAT AERONAUTICS



PROJECT OF ARISTOTLE SPACE & AERONAUTICS TEAM
HARDWARE DEVELOPMENT SUBSYSTEM

SENSORBOARD V3.0

PROJECT DEVELOPMENT
by
DENNIS DANIEL PLESSAS, ANASTASIOS KAZAKOS

August, 2024



1 Abstract

Firstly, I want to clarify that this report was generated by me after 2 years of my involvement in ASAT. Therefore, the programming part of the SensorBoard v3.0 contains only a personal STM32CubeIDE project since I was mainly involved in the PCB design.

One of the primary responsibilities of the Aeronautics team hardware development subsystem is the development of measurement devices. The purpose of these devices is to collect and process flight data, such as acceleration, altitude, pressure, and the like, which contribute significantly to the design of the aircraft. The SensorBoard v3.0 is the first printed circuit board ever designed and manufactured by the Aeronautics team. It should be noted that, unlike previous versions, where the measurement device was developed around a prototyping board, the SensorBoard v3.0 is significantly smaller and lighter since it is a PCB and therefore the design of the aircraft becomes further optimized. This particular PCB operates with an ARM microcontroller, like those used in Nucleo Boards, and its design was carried out with the help of the EasyEda editor. Furthermore, the microcontroller was programmed in the STM32CubeIDE environment and the libraries that were used were found on GitHub. In the figure below, the conceptual design for the development of SensorBoard v3.0 is illustrated. (1.1).

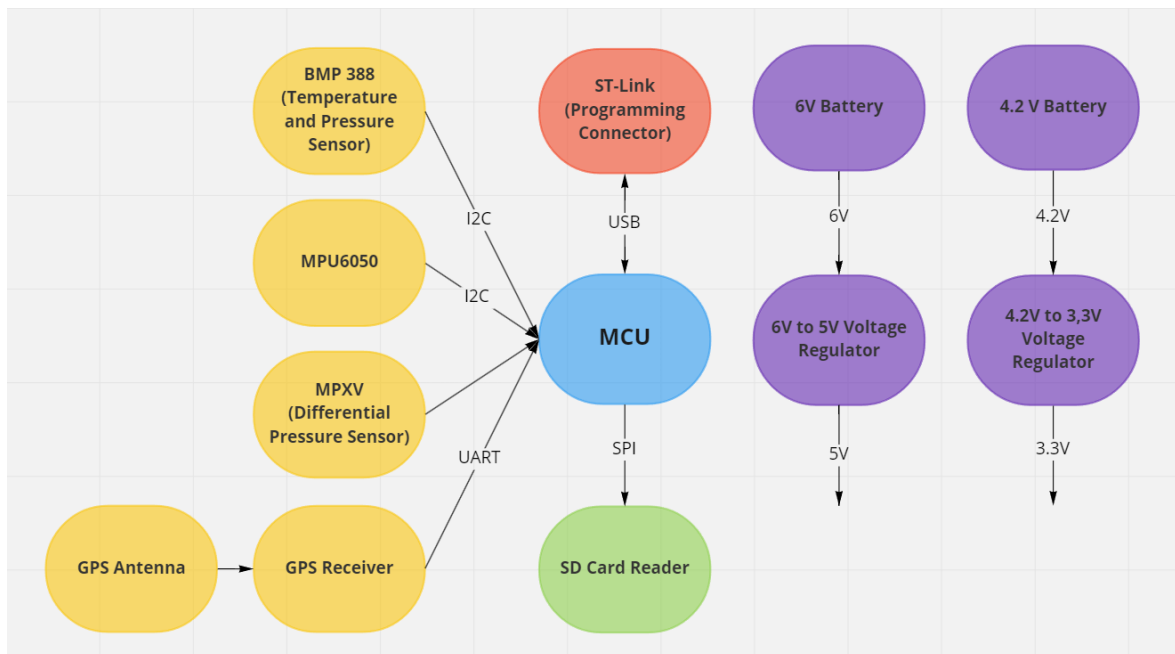


Figure 1.1: Conceptual Design.



Table of Contents

1 Abstract	2
2 Schematic	7
2.1 Power management	7
2.1.1 Voltage regulators	7
2.2 Microcontroller [STM32G0B1KET6N]	8
2.2.1 Pins for ST-Link Connection	10
2.2.2 Buttons	10
2.2.3 RGB LED	11
2.3 Sensors	12
2.3.1 Pressure and Temperature sensor [BMP-388]	12
2.3.2 Linear and Angular acceleration sensor [MPU-6050]	13
2.3.3 GPS Module [NEO-M8N-0]	15
2.3.4 Differential Pressure Sensor [MPXV7002DP]	17
2.4 Memory	18
2.4.1 Micro SD card reader [MEM2051-00-195-00-A]	18
2.5 Communication	19
2.5.1 Ground station [LoRa RA-02]	19
2.5.2 PCB communication protocols	20
2.5.2.1 UART protocol	20
2.5.2.2 I2C protocol	21
2.5.2.3 SPI protocol	21



2.6 PULL UP RESISTORS	22
3 Layout	24
4 My Personal Programming Project	26
4.1 HSV color model	26
4.2 Converting the HSV color model to RGB	27
4.3 Activated pins of the Microcontroller	28
4.4 Code of the Microcontroller	29



List of Figures

1.1	Conceptual Design.	3
2.1	Schematic design of the LP3871EMP-3.3/NOPB voltage regulator [1].	8
2.2	Schematic design of the LP3871EMP-5.0/NOPB voltage regulator [2].	8
2.3	Schematic design of the STM32G0B1KET6N microcontroller.	9
2.4	Header pins for the connection of the ST-Link.	10
2.5	(a) Schematic design of the reset button. (b) Schematic design of the button. [4]	11
2.6	Schematic design of the RGB LED.	11
2.7	Schematic design of the BMP-388 pressure and temperature sensor.	13
2.8	Schematic design of the MPU-6050 linear and angular acceleration sensor.	15
2.9	Schematic design of the NEO_M8N_0 GPS sensor.	16
2.10	Connector for the external GPS antenna.	17
2.11	Schematic design of the MPXV7002DP sensor.	18
2.12	Schematic design of the MEM2051-00-195-00-A micro SD card reader.	19
2.13	Schematic design of the LoRa RA-02 transceiver.	20
2.14	Pull-up resistors.	23
3.1	Layout design of the SensorBoard v3.0.	24
3.2	Picture of the SensorBoard v3.0 in operation.	25
4.1	Cone of the HSV color model.	26
4.2	Activated pins of the Microcontroller for the RGB project.	29



2 Schematic

2.1 Power management

The printed circuit board, SensorBoard v3.0, which was designed by the hardware development subsystem was powered by two batteries with voltages 4.2 V and 6 V. Using two voltage regulators, the necessary voltage levels of 3 V and 5 V were generated to power the components and facilitate their communication. It is worth mentioning that apart from the differential pressure sensor, for all other components, a 3.3 voltage was required for their operation and communication. Furthermore, the microcontroller that was used (STM32G0B1KET6N), was supporting communication in 3.3 V level, and therefore there was no need for level shifters between the components and the microcontroller.

2.1.1 Voltage regulators

The voltage regulators that were used were the LP3871EMP-3.3/NOPB and the LP3871EMP-5.0/NOPB for generating 3.3 V and 5 V, respectively. It is worth noting that in both cases, an LED was activated in order to indicate the successful operation of the voltage regulators. The schematic designs of the voltage regulators according to the datasheet of their manufacturer ([1] and [2] for the 4.2 V to 3.3 V regulator and 6 V to 5 V, respectively) are illustrated in the figures (2.1) and (2.2).

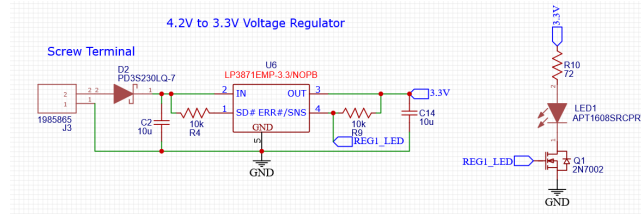


Figure 2.1: Schematic design of the LP3871EMP-3.3/NOPB voltage regulator [1].

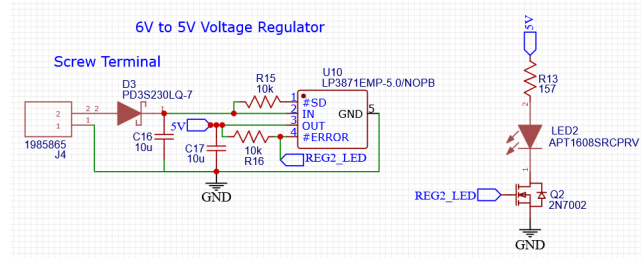


Figure 2.2: Schematic design of the LP3871EMP-5.0/NOPB voltage regulator [2].

2.2 Microcontroller [STM32G0B1KET6N]

The microcontroller that was used was the 32-bit ARM Cortex STM32G0B1KET6N with 32 pins operating at up to 64 MHz frequency.

Firstly, its supply voltage range is from 1.7 V to 3.6 V, while its temperature operation ranges from -40°C to 125°C . The STM32G0B1KET6N is also suitable for low power applications, since it has a comprehensive set of power saving modes, low power timers, and low power UART.

Moreover, it is equipped with a memory protection unit (MPU), including two high-speed embedded memories, a 144 Kbytes SRAM memory and up to 512 Kbytes of Flash memory. In order to fulfill the communication demands with the components, it supports three I2Cs, three SPIs / two I2S, and six USARTs. In addition, it offers one 12-bit ADC with up to 19 channels, one 12-bit DAC with two channels. The analytical details of the microcontroller can be found in the manufacturer's datasheet [3].



In table (1), the pinout of the microcontroller, presenting which pins are necessary for our application, is depicted. In figure (2.3), the

Table 1: Pinout of the STM32G0B1KET6N microcontroller.

Microcontroller pins	Connections	Microcontroller pins	Connections
1. PB9	MPU_SDA	17. PB15	LORA_RST
2. PC14-OSC32_IN	Not connected	18. PA8	LORA_CS
3. PC15-OSC32_OUT	Not connected	19. PA9	MPU_SCL
4. VDD/VDDA	3.3 V	20. VDDIO2	3.3 V
5. VSS/VSSA	Ground	21. PA10	BMP_SDA
6. PF2-NRST	RST	22. PA11[PA9]	BMP_SCL
7. PA0	PITOT	23. PA12[PA10]	SD_CS
8. PA1	BTN	24. PA13	SWDIO
9. PA2	GPS_RX	25. PA14-BOOT0	SWCLK
10. PA3	GPS_TX	26. PD0	Not connected
11. PA4	LORA_DIO	27. PD1	Not connected
12. PA5	SCK	28. PD2	Not connected
13. PA6	MISO	29. PD3	Not connected
14. PA7	MOSI	30. PB6	LED_G
15. PB0	Not connected	31. PB7	LED_B
16. PB1	Not connected	32. PB8	LED_R

schematic design of the microcontroller is illustrated.

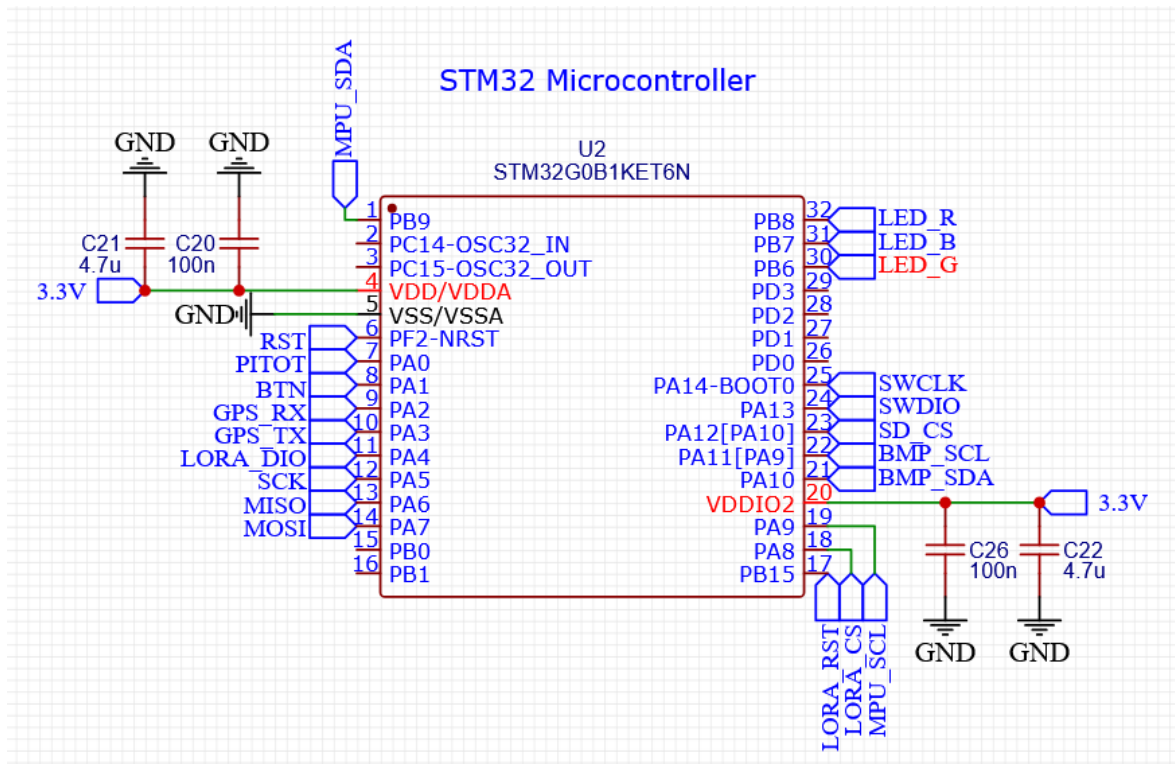


Figure 2.3: Schematic design of the STM32G0B1KET6N microcontroller.



2.2.1 Pins for ST-Link Connection

In figure (2.4), the pins for the connection of the ST-Link with the SensorBoard v3.0 are depicted. Two of them are responsible for the power of 3.3 V and ground, while the other two are for the clock signal (SWCLK) and the bidirectional data bus (SWDIO). The ST-Link is a programmer circuit that connects via a USB with a computer and via the aforementioned pins with the PCB, in order to program the microcontroller of the PCB.

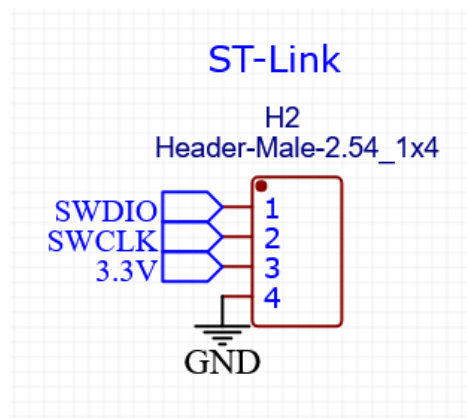


Figure 2.4: Header pins for the connection of the ST-Link.

2.2.2 Buttons

The SensorBoard v3.0 is featuring two buttons. Firstly, a reset button [PTS636-SM25-SMTR LFS] was connected to the PF2-NRST pin of the microcontroller, and its role was of great significance. During the flight testing, the objective was to complete as many flights as possible in our available time. Considering the limited time that we acquire, the hardware development subsystem decided to include a reset button in order to initialize the sensorboard for every new flight



without the need of a personal computer.

Furthermore, an additional button [PTS636-SM25-SMTR LFS] was decided to integrate into the PCB. This button was particularly useful in the lab testing, when the effectiveness of the code was being tested. To be more specific, a button command was inserted into certain code lines when a particular part of the code had to be thoroughly examined. In figure (2.5), the schematic design of each button is presented.

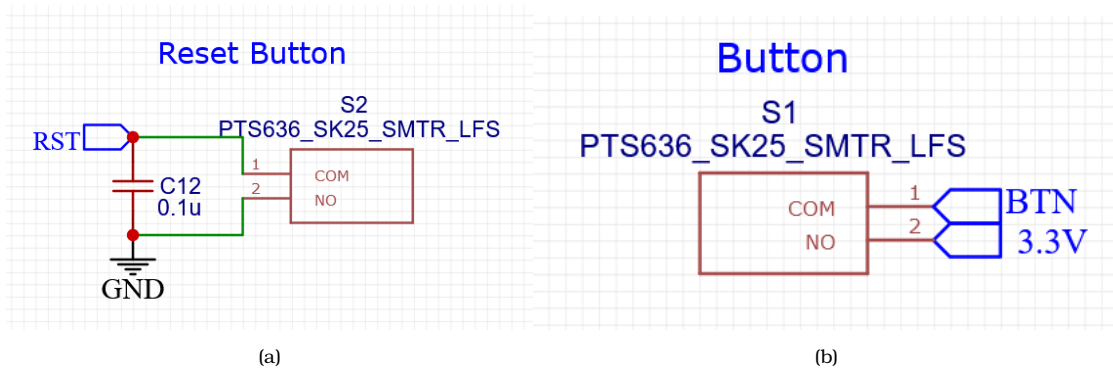


Figure 2.5: (a) Schematic design of the reset button. (b) Schematic design of the button. [4]

2.2.3 RGB LED

The RGB LED [APHF1608SEEQBDZGKC] was used in order to provide indication on which part of the code is being executed. In figure (2.6) the schematic design of the RGB LED is depicted.

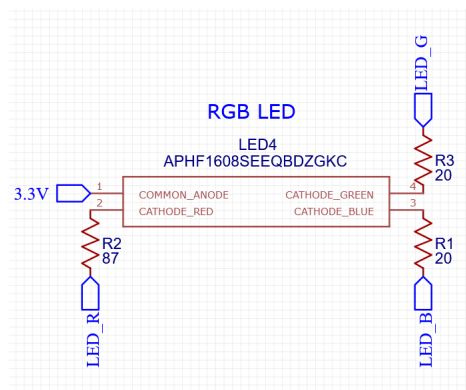


Figure 2.6: Schematic design of the RGB LED.



2.3 Sensors

2.3.1 Pressure and Temperature sensor [BMP-388]

The BMP-388 sensor provides the atmospheric pressure and temperature data at a certain altitude of flight. Furthermore, altitude was also calculated by developing a function that used the altitude pressure scale for an isothermal atmosphere for flights below 86 km. It should be mentioned that the BMP-388 is a digital sensor that uses the I²C protocol at 0.1 MHz to communicate with the microcontroller and its power voltage is 3.3 V. In table (2), the pinout of BMP-388 can be depicted. The wiring was based on the sensor's Datasheet ([6]), in

Table 2: Pinout of the BMP-388 pressure and temperature sensor [6].

BMP-388 pins	Connections
1. VDDIO	3.3 V
2. SCK	PA11
3. VSS_1	Ground
4. SDI	PA10
5. SDO	3.3 V
6. CSB	3.3 V
7. INT	Not connected
8. VSS_2	Ground
9. VSS_3	Ground
10. VDD	3.3V

which the I²C protocol was followed. The INT pin is intended for the use of interrupts, which we do not use, and therefore it is left unconnected. The SDO pin determines the least significant bit (LSB) of the sensor's I²C address. After verifying that the addresses of the other I²C devices (only the MPU6050 in this case) do not coincide with that of the BMP, this pin was connected to ground (VSS), although it could also have been connected to VDDIO. The complete wiring is shown in



Figure (2.7). It is worth noting that the pull-up resistors, as recommended by the datasheet, are not shown in the diagram below, since they are included in the level shifters paragraph.

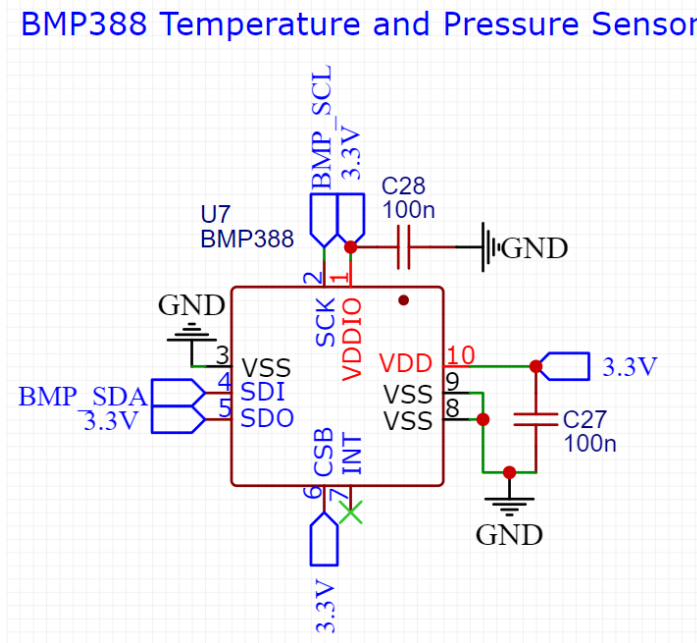


Figure 2.7: Schematic design of the BMP-388 pressure and temperature sensor.

2.3.2 Linear and Angular acceleration sensor [MPU-6050]

The MPU-6050 sensor provides the data of linear and angular acceleration of the aircraft. MPU-6050 is a digital sensor with a power voltage of 3.3 V, and it communicates with the microcontroller at a frequency of 0.4 MHz. The pinout of this particular sensor is depicted in table (3). The CLKIN pin can be connected to an external clock; otherwise, it is connected to ground. The AUX_DA and AUX_CL pins are used as auxiliary SDA and SCL pins. In our case, we do not need them, so we leave them unconnected. The VLOGIC and VDD pins are both power supply 3.3 V pins. The AD0 pin is used for the slave address of the protocol, which we leave unconnected. The REGOUT pin



Table 3: Pinout of the MPU-6050 linear and angular acceleration sensor [7].

MPU-6050 pins	Connections	MPU-6050 pins	Connections
1. CLKIN	Ground	14. NC	Not connected
2. NC	Not connected	15. NC	Not connected
3. NC	Not connected	16. NC	Not connected
4. NC	Not connected	17. NC	Not connected
5. NC	Not connected	18. GND	Ground
6. AUX_DA	Not connected	19. RESV	Not connected
7. AUX_CL	Not connected	20. CPOUT	Charge pump capacitor (C6)
8. VLOGIC	3.3V	21. RESV	Not connected
9. AD0	Ground	22. RESV	Not connected
10. REGOUT	Regulator filter capacitor (C5)	23. SCL	PA9
11. FSYNC	Ground	24. SDA	PB9
12. INT	Not connected	25. EP	Ground
13. VDD	3.3V		

is connected to the regulator filter capacitor, which, according to the datasheet, has a value of 100 nF. The FSYNC pin is used for frame synchronization, and since it is not used in our application, it is connected to the ground. The INT pin, which is used for interrupts, remains also unused, thus it is not connected. The CPOUT pin is connected to the charge pump capacitor, which is the capacitor C6 with a value of 2.2 nF. Lastly, the EP pin in the schematic is the metallic casing of the device, which is grounded.

It is worth mentioning that the MPU-6050's measurements of the linear acceleration are returned as a function of gravitational acceleration. Therefore, it is expected for the value of linear acceleration in the axis perpendicular to the Earth's surface to tend towards the value of 1g, while the two remaining perpendicular axes towards zero when the aircraft is flying in parallel with the earth's surface. In figure (2.8), the schematic representation of the MPU-6050 sensor is presented.

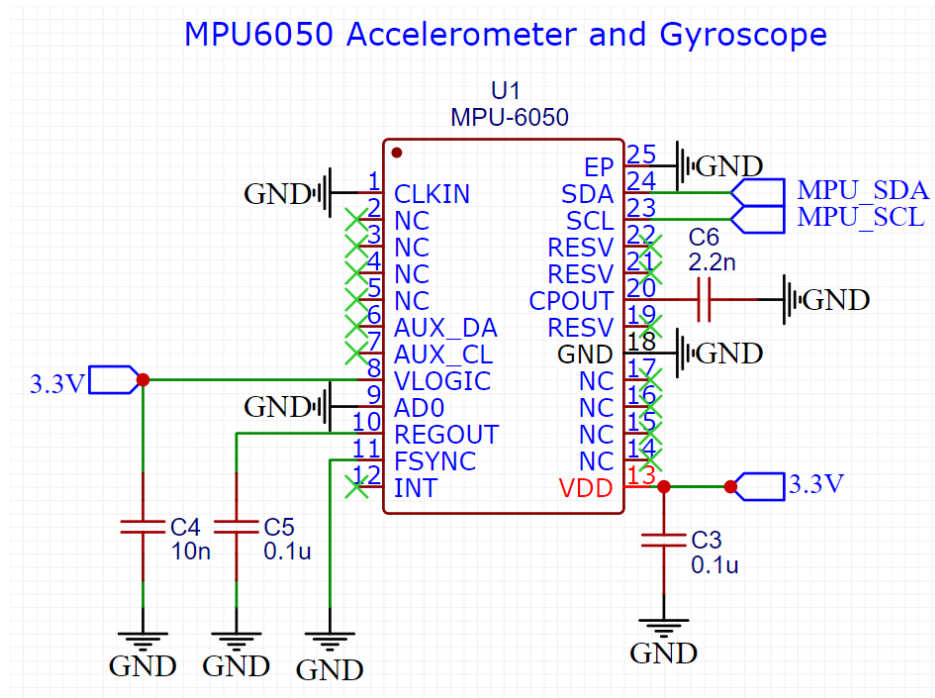


Figure 2.8: Schematic design of the MPU-6050 linear and angular acceleration sensor.

2.3.3 GPS Module [NEO-M8N-0]

The data for the exact location of the aircraft, as well as its altitude, are collected with the help of the digital sensor NEO-M8N-0. The NEO-M8N-0 communicates with the microcontroller in accordance with the UART communication protocol. Furthermore, the power supply voltage of the GPS module is 3.3 V, and additionally, it features a backup voltage (V_BCKP). The role of the backup voltage is to be activated for a period of time before the operation of the GPS module, in order for the module to be properly configured and the accuracy of the data to be improved. Moreover, the External Antenna Connector, where the antenna is connected, is wired to the RF_IN pin of the module. In the table (4) the pinout of the digital sensor NEO-M8N-0 is depicted.

In figure (2.9) the schematic design of the sensor is illustrated,



GPS Module pins	Connections	GPS Module pins	Connections
1. SAFEBOOT_N	Not connected	13. GND_3	Ground
2. D_SEL	Not connected	14. RE-SERVED_1	Not connected
3. TIMEPULSE	Not connected	15. RE-SERVED_2	Not connected
4. EXTINT	Not connected	16. RE-SERVED_3	Not connected
5. USB_DM	Not connected	17. RE-SERVED_4	Not connected
6. USB_DP	Not connected	18. SDA_/_SPI_CS_N	Not connected
7. VDD_USB	Ground	19. SCL_/_SPI_SLK	Not connected
8. RESET_N	Not connected	20. TXD_/_SPI_MISO	PA2
9. VCC_RF	Not connected	21. RXD_/_SPI_MOSI	PA3
10. GND_1	Ground	22. V_BCKP	3.3 V
11. RF_IN	SIG	23. VCC	3.3 V
12. GND_2	Ground	24. GND_4	Ground

GPS module

IC4
NEO-M8N-0

3.3V

C1
100n

GND

GPS TX
GPS RX

3.3V

ANT1

13 GND
14
15
16
17
18
19
20
21
22
23
24

GND_3
RESERVED_1
RESERVED_2
RESERVED_3
RESERVED_4
SDA/_SPI_CS_N
SCL/_SPI_SLK
TXD/_SPI_MISO
RXD/_SPI_MOSI
V_BCKP
VCC
GND_4

12 GND
11
10
9
8
7
6
5
4
3
2
1

GND_2
RF_IN
GND_1
VCC_RF
RESET_N
VDD_USB
USB_DP
USB_DM
EXTINT
TIMEPULSE
D_SEL
SAFEBOOT_N

Furthermore, an external GPS Antenna connector was integrated in the PCB, in order for the GPS module to detect the position coor-



dinates of the aircraft. The antenna connector is illustrated in figure (2.10).

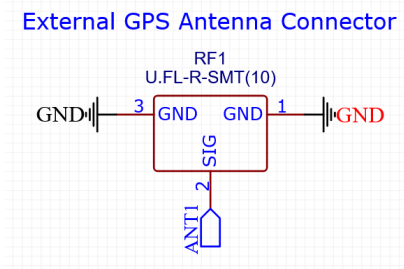


Figure 2.10: Connector for the external GPS antenna.

2.3.4 Differential Pressure Sensor [MPXV7002DP]

The sensor that was used to provide a supplementary pressure measurement was the analog sensor MPXV7002DP, which calculates the differential pressure. This sensor has two nozzles to which silicone tubes are connected, and these in turn are connected to the Pitot tube. In particular, the sensor returns a value in the range of 0-512, which is converted in the code to voltage using the relation $V_{out} = \frac{5}{512} \cdot sensorValue$. The differential pressure is determined through the voltage, utilizing the transfer function. Knowing the pressure at the two nozzles and using Bernoulli's equation, the speed of the aircraft can be calculated. Furthermore, in table (5), it can be depicted that the sensor has an analog pin, so decoupling capacitors are required according to the datasheet. In addition, resistors (R19) and (R18) are necessary to convert the 5 V voltage to 3.3 V in order to communicate the MPXV7002DP with the microcontroller. In table (5), the pinout of the differential pressure sensor is depicted, according to the datasheet ([9]).



Table 5: Pinout of the MPXV7002DP differential pressure sensor [9].

MPXV7002DP pins	Connections	MPXV7002DP pins	Connections
1. N/C	Not connected	5. N/C	Not connected
2. Vs	5 V	6. N/C	Not connected
3. GND	Ground	7. N/C	Ground
4. Vout	Charge pump capacitor (C9)	8. N/C	Not connected

In figure (2.11) the schematic design of the sensor is illustrated.

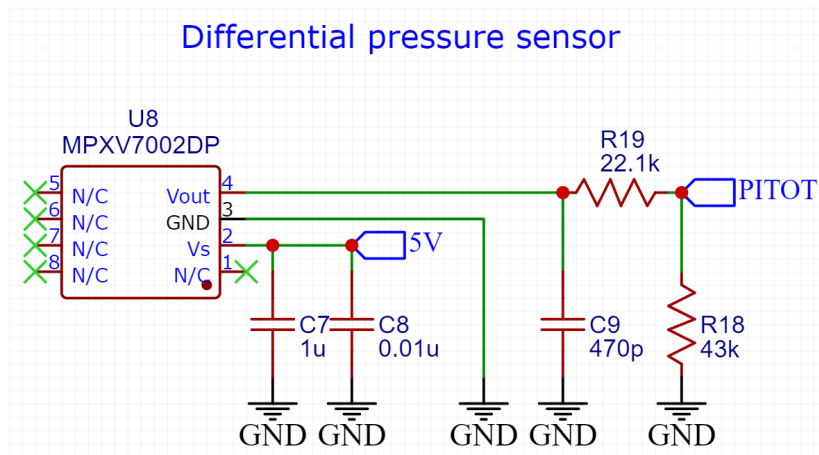


Figure 2.11: Schematic design of the MPXV7002DP sensor.

2.4 Memory

2.4.1 Micro SD card reader [MEM2051-00-195-00-A]

The data, which were collected during a flight, were stored in a micro SD card. Therefore, a micro SD card reader had to be included in our PCB, specifically the MEM2051-00-195-00-A. The MEM2051-00-195-00-A card reader supports the SPI communication protocol. In table (6), the pinout of the card reader is depicted, according to the Datasheet ([10]).

In figure (2.12) the schematic design of the card reader is illustrated.



Table 6: Pinout of MEM2051-00-195-00-A micro SD card reader [10].

GPS Module pins	Connections	GPS Module pins	Connections
1. DAT2	Not connected	8. DAT1	Not connected
2. CD/DAT3	PA12	9. MP1	Ground
3. CMD	PA7	10. MP2	Ground
4. VDD	3.3 V	11. MP3	Ground
5. CLK	PA5	12. MP4	Ground
6. VSS_DP	Ground	13. MP5	Ground
7. DAT0	PA6		

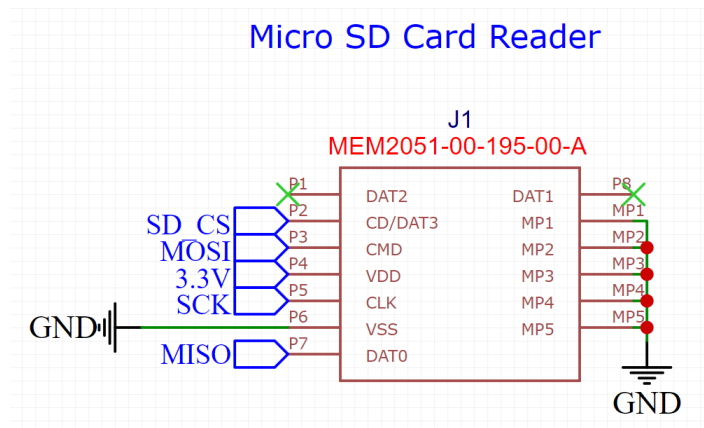


Figure 2.12: Schematic design of the MEM2051-00-195-00-A micro SD card reader.

2.5 Communication

2.5.1 Ground station [LoRa RA-02]

For the communication of the SensorBoard v3.0 with a ground station, LoRa RA-02 was embedded in order to have live monitoring of the most crucial data. The frequency at which the transmission takes place, it ranges from 410 to 525 MHz, and the communication protocol used for the interaction with the microcontroller is the SPI. In table (7), the pinout of the LoRa RA-02 is depicted.

Moreover, the schematic design of the LoRa RA-02 is illustrated in figure (2.13).



Table 7: Pinout of LoRa RA-02 transceiver.

RA-02 module pins	Connections	RA-02 module pins	Connections
1. GND	Ground	9. GND	Ground
2. GND	Ground	10. DIO4	Not connected
3. 3.3 V	3.3 V	11. DIO5	Not connected
4. RESET	LORA_RST	12. SCK	SCK
5. DIO0	LORA_DIO	13. MISO	MISO
6. DIO1	Not connected	14. MOSI	MOSI
7. DIO2	Not connected	15. NSS	LORA_CS
8. DIO3	Not connected	16. GND	Ground

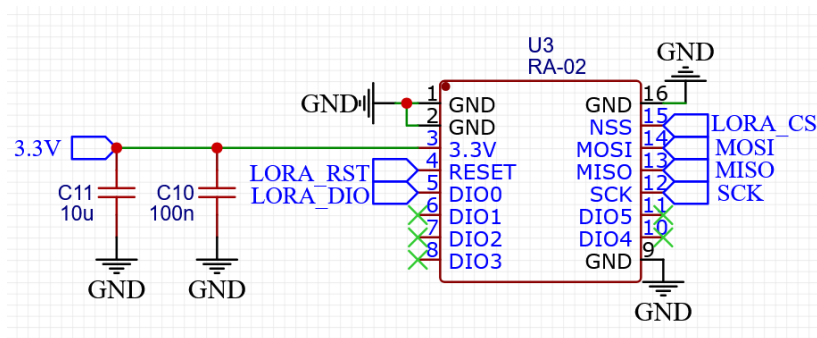


Figure 2.13: Schematic design of the LoRa RA-02 transceiver.

2.5.2 PCB communication protocols

For the transmission of the digital data between the components, three types of serial communication were used. To be more specific, the GPS module communicates with the microcontroller via the UART protocol, the BMP388 and the MPU6050 via the I²C protocol, and finally, the Micro SD card reader and the LoRa RA-02 via the SPI protocol. It has to be mentioned that all three types of communication are serial, meaning that the data bits are transferred in series rather than in parallel. In the following paragraphs, a description of the protocols operation is being presented.

2.5.2.1 UART protocol In the UART protocol, the use of two wires are necessary for its operation. In particular, the transmitting and receiv-



ing data are transferred in different wires, named TX and RX, respectively. Since this type of communication is asynchronous, a common configuration should exist between the transmitter and the receiver. To be more exact, the three settings that should be identical both to the transmitter and the receiver are:

- The transmission speed.
- The data length.
- The start and stop bit.

2.5.2.2 I2C protocol This communication protocol is synchronous, and it is widely used for attaching lower speed peripheral ICs to processors and microcontrollers in short distance communications. In this case, similar to the previous protocol, two wires are needed; however, one is only for data transmission [SDA] and the other for the clock signal [SCL]. Since a clock signal is present, there is no need to know the transmission speed of the data. Furthermore, it should be mentioned that many components can use the same wires for communication with the microcontroller. The reason is that since every slave device has a particular address, the microcontroller sends the address of the slave device in the SDA wire in order to establish their communication and afterwards the data are transferred.

2.5.2.3 SPI protocol In this particular protocol, four wires are required for the communication of the PCB devices. Specifically, the wires are



the following:

- SCLK: The wire for the clock signal.
- MOSI (Master Output Slave Input): The wire responsible for data transmission from the master device to the slave device.
- MISO (Master Input Slave Output): The wire responsible for data transmission from the slave device to the master device.
- CS (Chip Select): The wire responsible for the selection of the slave device with which the microcontroller is about to communicate.

The great advantage of this type of communication over the I²C is that the data transmission is duplex. In particular, a device can send data while receiving data. Furthermore, the main characteristics of each aforementioned protocol are presented in table (8), where the differences between them are apparent.

Table 8: Main characteristics of the UART, I²C and SPI protocols.

	UART	I ² C	SPI
Data transmission	1 to 1	Simplex	Duplex
Clock	Asynchronous	Synchronous	Synchronous
Wires	2	2	4
Transmission speed	20 Kbps	1 Mbps	25 Mbps
Acknowledge pins	YES	NO	NO
Power	Medium	Medium	Low
Distance	15 m	1 m	20 cm

2.6 PULL UP RESISTORS

According to the datasheets of the components, there is a need of pull up resistors in certain wires. The values of these resistors were determined by the datasheet of each component. In the figure (2.14),



the pull-up resistors are illustrated.

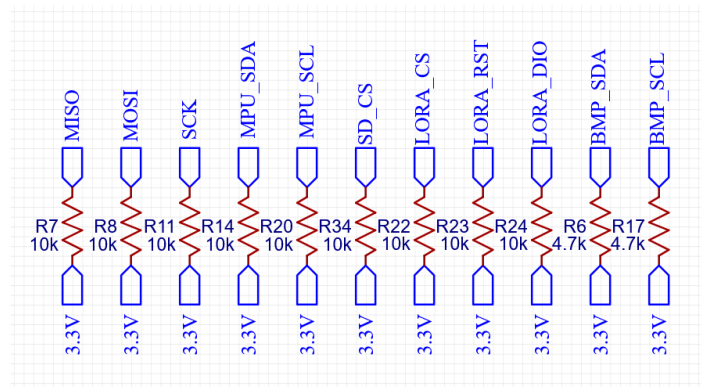


Figure 2.14: Pull-up resistors.



3 Layout

The layout design of the printed circuit board was developed in the EasyEDA environment. After placing the components on the PCB and routing them manually, the design rule check was applied. The design rules that had to be met in our design were defined by the board manufacturer, JLCPCB. After making the necessary corrections in our design, the SensorBoard v3.0 was ordered. In figure (3.1), the layout design of the SensorBoard v3.0 is depicted, where the main devices are marked for a better understanding of the design. Furthermore, in

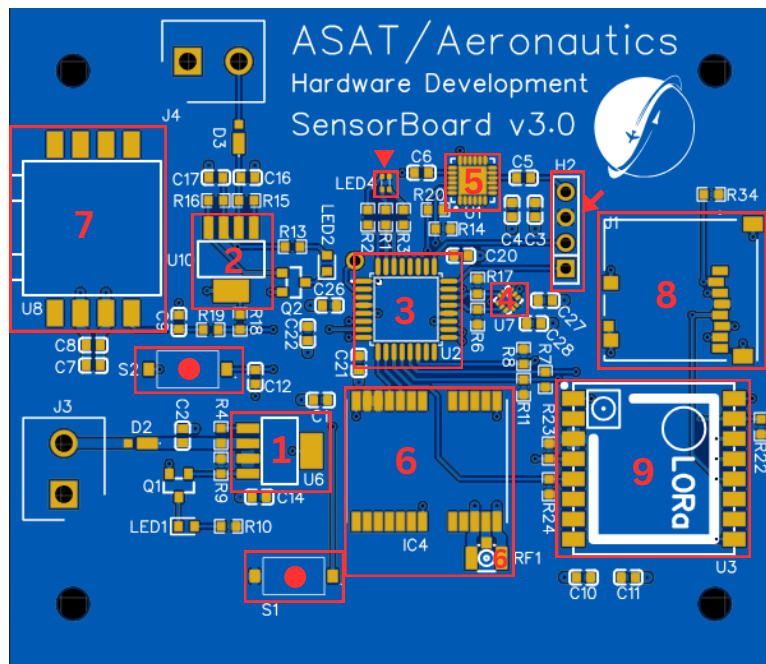


Figure 3.1: Layout design of the SensorBoard v3.0.

table (9), the mapping of the SensorBoard's v3.0 devices for the above figure (2.1) is depicted.

Finally, a figure illustrating the operation of SensorBoard v3.0 is presented above (3.2). It is clear that only the 3.3 V pin and the ground pin is connected to the SensorBoard v3.0, because it is already



Table 9: Mapping of the SensorBoard's v3.0 devices to the figure (2.1).

Numbering	Devices	Numbering	Devices
1	Voltage regulator (4.2 V to 3.3 V)	7	Differential pressure sensor
2	Voltage regulator (6 V to 5 V)	8	Card reader
3	Microcontroller	9	LoRa
4	BMP388	circle	Buttons
5	MPU6050	Triangle	RGB LED
6	GPS Module & Antenna	Arrow	ST-Link connector

programmed to change the color of its RGB LED periodically. This is a personal project for the Thessaloniki International Fair of 2022.

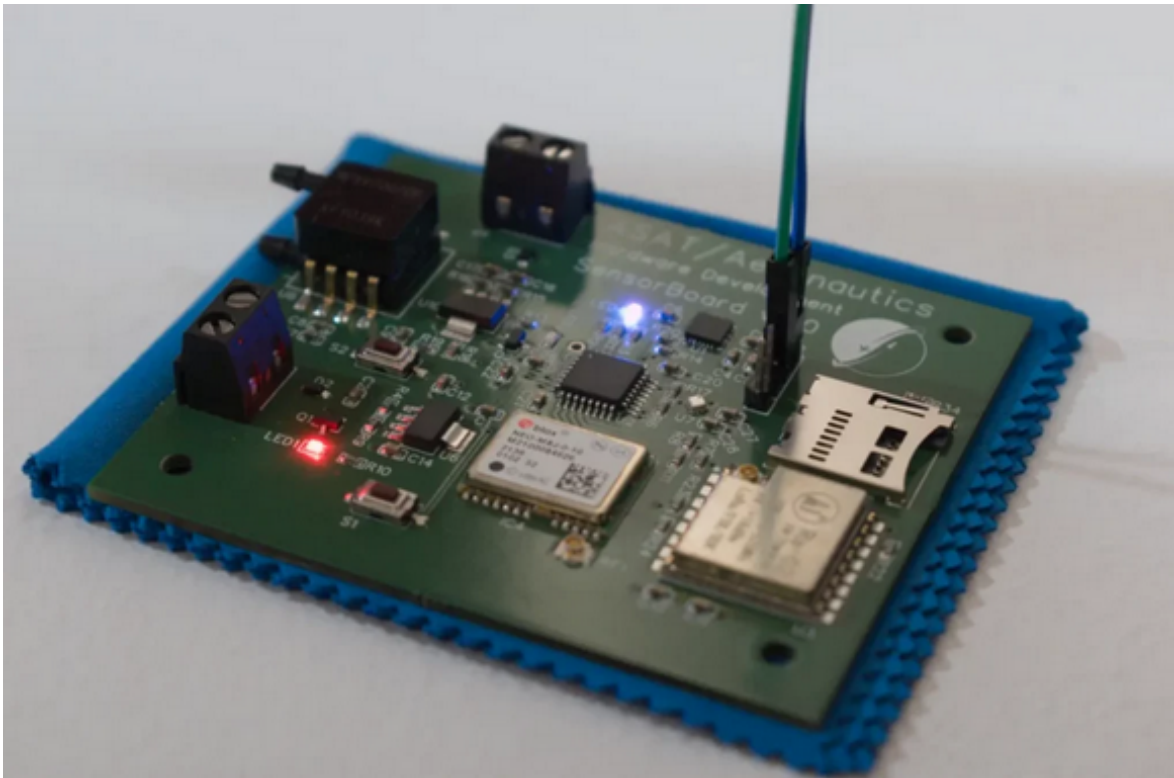


Figure 3.2: Picture of the SensorBoard v3.0 in operation.



4 My Personal Programming Project

The code generation for the detection of important variables was entirely covered by my colleague "Dennis Daniel Plessas", whom I really thank for his great impact in our project.

In this section of the report, a small programming project that was generated by me is going to be analyzed. The purpose of this project was to make the RGB LED of the SensorBoard v3.0 change color periodically during a presentation of our board in the Thessaloniki International Fair, in September 2022. In order to accomplish this goal, the use of the HSV model for representing the color is found to be more appropriate. This is because, only one coordinate is needed to change in order to change the actual color of the LED.

4.1 HSV color model

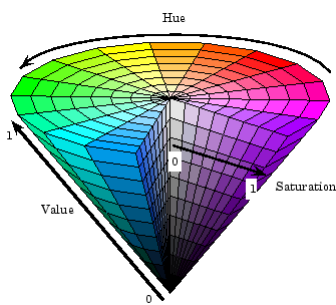


Figure 4.1: Cone of the HSV color model.

In contrast to the RGB color model, where three colors are added together in different quantities in order to produce a color, the HSV color model is based on the dimensions of the color as it is depicted in the figure (4.1). To be more specific, the coordinates of this model remain three; however, they represent the coordinates of a color cone, where the radius of the cone is the Saturation, the Value is the height of the cone, and the Hue is the angle counterclockwise around the cone. In particular, each coordinate represent:



- Hue : It represents the actual color, and its value ranges from 0^0 to 360^0 .
- Value : It represents the brightness of the color and ranges from 0.0 to 1.0, where 0.0 is black and 1.0 is white.
- Saturation : It represents the vibrancy of the color and ranges from 0.0 to 1.0, where 0.0 is a shade of gray and 1.0 is the full color.

4.2 Converting the HSV color model to RGB

It has already mentioned that the reason behind the use of the HSV model is the convenience of changing only one coordinate in a for loop that it will be presented soon. However, we need to translate the HSV color model to the RGB model to activate the RGB LED. Therefore, the following equations have been used for the conversion (these equations came from a website, rather than a scientific paper; however, since they presented an appropriate outcome the validation of them was not examined). Firstly, the $M = \max\{R, G, B\}$ and $m = \min\{R, G, B\}$ were associated with the Value and Saturation coordinates as:

$$M = 255 * v \quad (1)$$

$$m = M * (1 - S) \quad (2)$$



Moreover, the next number was needed to be calculated (where mod_2 is the division modulo 2):

$$z = (M - m) * [1 - |(H/60)mod_2 - 1] \quad (3)$$

Afterwards, with the use of M , m , and z parameters the R, G, B coordinates of the RGB color model can be calculated for the six spectrums of 60^0 in the range of 0^0 to 360^0 of Hue.

4.3 Activated pins of the Microcontroller

In figure (4.2), the pins which are involved in this project are depicted. In particular, the SYS_SWCLK and the SYS_SWDIO are used for the programming of the microcontroller, while the pins PB8, PB7, and PB6 are in PWM mode of the channels CH1, CH2, and CH3 of the timer TIM4. It has to be mentioned that the RGB LED is controlled with Pulse-Width Modulation. According to the “Designing Embedded Systems with PIC Microcontrollers” book [12]; Pulse-width modulation is “a digital technique to control a signal by repeatedly toggling a signal between a HIGH and a LOW state in a consistent pattern. We can portray new information by changing how long the signal is HIGH versus LOW. The PWM signal has two key parameters—frequency and duty cycle”. Since the TIM4 of STM32G0B1KET6N microcontroller is a 16-bit timer the counter period can be set between 0 and 65535. The period of the TIM4 was 2500 and no prescalers were used.

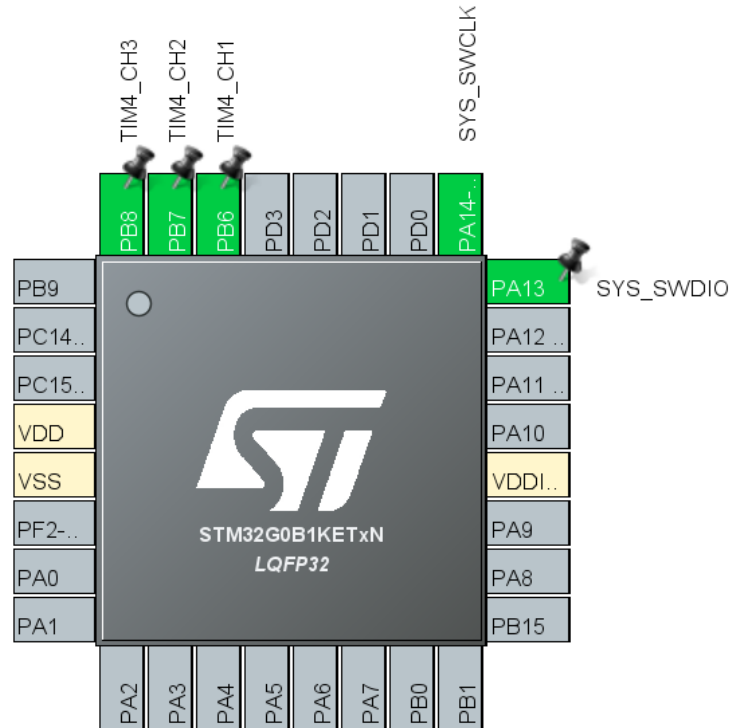


Figure 4.2: Activated pins of the Microcontroller for the RGB project.

4.4 Code of the Microcontroller

In this section the code that was generated by the user is presented, since the initialization of the pins and the system's clock is made by the Cube MX software. Firstly, the libraries that were included are being presented in listing [1]. The `<math.h>` was necessary in order to use the function `fmodf`.

```
1 /* Includes ----- */
2 #include "main.h"
3
4 /* Private includes ----- */
5 /* USER CODE BEGIN Includes */
6 #include <stdlib.h>
7 #include <math.h>
8 /* USER CODE END Includes */
```

Listing 1: Libraries that were used.



Afterwards, some important parameters were defined (listing [2]), such as the time period of the clock, the Saturation and the Value coordinates, and three coefficients. The Saturation and the Value was chosen according to our personal preference, since these coordinates do not affect the actual color. The red, blue, and green coefficients were used to enhance the quality of the color. We noticed that the RGB LED had more intense green and red light rather than blue, and therefore we reduced the intensity of these two colors to have a better distinction among the colors.

```
1 /* Private define ----- */
2 /* USER CODE BEGIN PD */
3 #define TIM4_PERIOD 2500
4 #define RED_COEF 0.75
5 #define GREEN_COEF 0.5
6 #define BLUE_COEF 1.0
7 #define S_COEF 0.9 // 0 <= S_COEF <= 1 for Saturation
8 #define V_COEF 0.9 // 0 <= V_COEF <= 1 for Value
9 /* USER CODE END PD */
```

Listing 2: Definition of important parameters.

The next step was to develop a function, which would convert the HVS color to the RGB color for the LED. Furthermore, it had to set the duty cycle of each PWM channel. Controlling in what percentage of a full PWM cycle the signal of each color is high, we can determine the proportion of red, blue, and green that we have to add in order to produce every possible color that can be represented with the RGB color model. In the first ten lines of code of the listing [3], it is apparent that the parameters that were defined in subsection [4.2] are declared,



while the variables red, blue, and green are the colors of the RGB model.

```
1  /* Private user code -----*/
2  /* USER CODE BEGIN 0 */
3  void ChangeColor (uint16_t h, float s, float v)
4  {
5      float red;
6      float green;
7      float blue;
8      float M = 255.0 * v;
9      float m = M * (1 - s);
10     float z = (M - m) * (1 - abs(fmodf((float)h / 60.0, 2.0) - 1));
```

Listing 3: ChangeColor function.

For the next 36 lines (listing [4]) there are some if statements in which the variables red, blue, and green are being declared, depending on the value of the Hue. The equation inside the if statements was also provided by the website ([13]).

```
1  if ((h>=0) && (h<60))
2  {
3      red = M;
4      green = z + m;
5      blue = m;
6  }
7  else if ((h>=60) && (h<120))
8  {
9      red = z + m;
10     green = M;
11     blue = m;
12 }
13 else if ((h>=120) && (h<180))
14 {
15     red = m;
16     green = M;
17     blue = z + m;
18 }
```



```
19  else if ((h>=180) && (h<240))
20  {
21      red = m;
22      green = z + m;
23      blue = M;
24  }
25  else if ((h>=240) && (h<300))
26  {
27      red = z + m;
28      green = m;
29      blue = M;
30  }
31  else if ((h>=300) && (h<360))
32  {
33      red = M;
34      green = m;
35      blue = z + m;
36  }
```

Listing 4: ChangeColor function.

After the execution of the if statements, the duty cycle of each channel was declared by storing a value between 0 and 2500 (the time period) in the three TIM4 registers for each channel (listing[5]). The operation (color / 255.0) is used for the normalization of the RGB coordinate in the range of 0.0 to 1.0.

```
1  TIM4->CCR3 = (red / 255.0) * RED_COEF * TIM4_PERIOD;    // Red channel
2  TIM4->CCR1 = (green / 255.0) * GREEN_COEF * TIM4_PERIOD; // Green channel
3  TIM4->CCR2 = (blue / 255.0) * BLUE_COEF * TIM4_PERIOD;  // Blue channel
4  }
5  /* USER CODE END 0 */
```

Listing 5: ChangeColor function.

Lastly, inside the main function the PWM signal generation on Timer TIM4 for the three channels was started and a for loop was generated



that executes the ChangeColor function for the S_COEF, V_COEF that we defined, and for the integral values of Hue, from 0^0 to 360^0 (listing [6]). Moreover, an if statement was incorporated into the for loop to reset the Hue to 0^0 when it reaches 360^0 , enabling the RGB LED to run continuously.

```
1  /* USER CODE BEGIN 2 */
2  HAL_TIM_PWM_Start (&htim4, TIM_CHANNEL_1);
3  HAL_TIM_PWM_Start (&htim4, TIM_CHANNEL_2);
4  HAL_TIM_PWM_Start (&htim4, TIM_CHANNEL_3);
5  uint16_t H;
6  for (H = 0 ; H <= 360 ; H++)
7  {
8      ChangeColor (H, S_COEF, V_COEF);
9      if (H == 360)
10     {
11         H = 0;
12     }
13 }
14 /* USER CODE END 2 */
```

Listing 6: ChangeColor function.



References

- [1] LP3871EMP-3.3/NOPB voltage regulator: Datasheet
- [2] LP3871EMP-5.0/NOPB voltage regulator: Datasheet
- [3] STM32G0B1KET6N microcontroller: Datasheet
- [4] Button: Datasheet
- [5] RGB LED: Datasheet
- [6] BMP-388 pressure and temperature sensor: Datasheet
- [7] MPU-6050 linear and angular acceleration sensor: Datasheet
- [8] NEO_M8N_0 GPS module: Datasheet
- [9] MPXV7002DP differential pressure sensor: Datasheet
- [10] MEM2051-00-195-00-A card reader: Datasheet
- [11] LoRa RA-02 transceiver: Datasheet
- [12] Designing Embedded Systems with PIC Microcontrollers : Book
- [13] Formulas for the HSV to RGB color model conversion : URL